Group Project: A Command-Line Campus Room Booking System

Department of Computing

11th September 2026

Deadlines

1. Form your groups of 3 to 4 students before 16:00, 2nd October 2026. Afterward, students with no group will be randomly grouped. Requests for group change will be approved only if written agreements from all members of the involved groups are provided before 14:00, 9th October 2026.
2. Compress the source code (including the tests), the tests, the user manual, the presentation slides and recording, and the project report into a single ZIP archive and submit it on Blackboard before 14:00, 22nd November 2026.
3. Submit the design review reports in PDF or JPG format on Blackboard before 14:00, 28th November 2026.
4. Late submissions will lead to 25% deduction of points per day.

1 Overview

The goal of this project is to develop a command-line Campus ROom Booking System (Crobs) in Java. Crobs allows users to manage university rooms and reservations. Two types of users are supported: **admin users** and **regular users**. All types of users can make, modify, and delete room bookings for themselves, as well as search for available rooms at specific timeslots with certain desired features. Admin users can additionally:

- Add, modify, and delete rooms
- Add and remove regular users

There is also a unique **superadmin** user, a type of admin user, and the only user who can modify the access privileges of other users. When Crobs starts, the system automatically creates a superadmin user.

All types of users can also view the current weekly schedule for a specific room, but only admin users can see which user has booked the room.

2 Requirements

The Crobs tool should provide a command line interface (CLI) to its users, and all rooms and users have *unique* IDs.

[REQ1] (3 points) All of the operations executed during a Crobs session should be logged into two files. The first file is in HTML format, where the commands are recorded in a table, and each row of the table has two columns, one for the operation index

and the other for the operation command. The second file is in plain TXT format, where the commands are recorded in execution order, and each line contains one command. The names of the two files should be input as parameters when launching the CROBS tool, e.g., `java hk.edu.polyu.comp.comp2021.crobs.Application -html d:\log.html -txt d:\log.txt`.

[REQ2] (1.5 points) The system should allow admin users to add a new (regular) user.
Command: `addUser` *name*
Effect: Creates a new regular user with the given name. The system should automatically generate a unique user ID for the new user.

[REQ3] (1.5 points) The system should allow admin users to remove an existing (regular) user.
Command: `removeUser` *targetUserID*
Effect: Removes the regular user with ID targetUserId. Removing a user also cancels all bookings made by that user.

[REQ4] (2 points) The system should allow the superadmin user to modify the access privileges of other users.
Command: `changeRole` *targetUserId newRole*
Effect: Changes the role of the target user to *newRole*.

[REQ5] (1.5 points) The system should support logging in as an existing user.
Command: `login` *userID*
Effect: Logs in as the user with the specified user ID. At the beginning of each CROBS session, no user is logged in.

[REQ6] (1 points) The system should support logging out the current user.
Command: `logout`
Effect: Logs out the currently logged-in user. After logout, no user is logged in.

[REQ7] (2 points) The system should allow admin users to add a new room.
Command: `addRoom` *roomType capacity features*
Effect: Creates a new room with the given room type, capacity, and features. The supported room types are: lectureTheatre, computerLab, seminarRoom, and meetingRoom. Features should be represented as a comma-separated list without spaces. The system should automatically generate a unique room ID for the new room.

[REQ8] (2 points) The system should allow admin users to modify room information.
Command: `modifyRoom` *roomID field newValue*
Effect: Modifies one field of the specified room.

[REQ9] (1.5 points) The system should allow admin users to delete a room.
Command: `deleteRoom` *roomID*
Effect: Deletes the room with the specified room ID. Deleting a room also cancels all future bookings for that room.

[REQ10] (4 points) The system should support creating a room booking for the currently logged-in user.
Command: `book` *roomId date startTime endTime purpose*
Effect: Creates a booking for the currently logged-in user. The system should automatically generate a unique booking ID. Two bookings conflict if they refer to the same room on the same date and their time intervals overlap.

[REQ11] (3.5 points) The system should support modifying an existing booking owned by the currently logged-in user.
Command: `modifyBooking` *bookingID roomId date startTime endTime purpose*
Effect: Updates the room, date, start time, end time, and purpose of the specified booking. The booking ID and owner user ID should remain unchanged.

[REQ12] (2.5 points) The system should support deleting or cancelling an existing booking owned by the currently logged-in user.
Command: `deleteBooking` *bookingID*
Effect: Deletes or cancels the booking with the specified booking ID.

[REQ13] (3.5 points) The system should support searching for available rooms at a specific timeslot with certain desired features.
Command: `search` *date startTime endTime minCapacity features*
Effect: Lists all rooms that have have capacity greater than or equal to minCapacity, contain all required features, and are available during the specified time slot.

[REQ14] (3.5 points) The system should support viewing the weekly schedule for a specific room.
Command: `schedule` *roomId weekStartDate*
Effect: Lists all active bookings for the specified room in the 7-day period starting from weekStartDate, sorted by date and start time. Regular users can only see the userID corresponding to a booking if it is their own booking. Admin users can the userIDs corresponding to all bookings.

[REQ15] (1.5 points) The system should support listing information about a room.
Command: `listRoom` *roomID*
Effect: Displays the room ID, room type, capacity, and features.

[REQ16] (1.5 points) The system should support listing all rooms.
Command: `listAllRooms`
Effect: Lists all rooms and their room type, capacity, and features. The list should be sorted in ascending order of room ID.

[REQ17] (1.5 points) The system should support listing all bookings made by the current user.
Command: `listBookings`
Effect: Lists all bookings made by the current user. Bookings should be sorted by date and start time.

[REQ18] (1 points) The user should be able to terminate the current execution of Crobs.
Command: `quit`
Effect: Terminates the program.

The Crobs tool that you need to implement in this project will discard the current instance when it is terminated. Support for persistent storage can be added to Crobs in the future, but you do *not* need to take this into account when designing and implementing the system in this project. That said, you *do* need to use your best judgment to gracefully handle situations where a command is ill-formed or a required action cannot be accomplished, e.g., because the user lacks the necessary access privileges or an ID is not defined. Poor design in handling those situations will lead to point deductions.

The Crobs tool may be extended with the following *bonus* features:

[BON1] (6 points) Support for a graphical user interface (GUI) to perform and depict the above commands. Also note that you still need to implement the command line interface even if your tool can run in the GUI mode.

[BON2] (3 points) Support for the `undo` and `redo`[1] of all commands except `login`, `logout`, `search`, `listRoom`, `listAllRooms`, `listBookings`, and `quit`.

[1] https://en.wikipedia.org/wiki/Undo

3 Group Forming

This is a group project. Each group may have 3 to 4 members from the same class session, i.e., *no group across different class sessions is allowed*. Form your groups before 14:00, 2nd October 2026. Afterward, students with no group will be randomly grouped. Requests for group change will be approved only if written agreements from all members of the involved groups are provided before 14:00, 9th October 2026.

4 Code Inspection

The inspection tool of IntelliJ IDEA[2] checks your program to identify potential problems in the source code. We have prepared a set of rules to be used in grading (`COMP2021_PROJECT.xml` in the project material package). Import the rules into your IntelliJ IDEA IDE and check your implementation against the rules. Unit tests do not need to be checked.

The inspection tool is able to check for many potential problems, but only a few of them are considered errors by the provided rule set. One point will be deducted for each *error* in your code reported by the tool.

5 Line Coverage of Unit Tests

The line coverage[3] achieved by a suite of unit tests is the percentage of source code lines that the tests exercise, and it is an important measure of the test suite's quality. The higher the coverage, the more thoroughly the suite of tests exercised a program.

You should follow the Model-View-Controller (MVC) pattern[4] in designing the CROBS tool. Put all Java classes for the model into package `hk.edu.polyu.comp.comp2021.crobs.model` (see the structure in the sample project) and write tests for the model classes.

During grading, we will use the coverage tool of IntelliJ IDEA[5] to get the line coverage of your tests on package `hk.edu.polyu.comp.comp2021.crobs.model`. You will get 10 base points for line coverage between 90% and 100%, 9 base points for coverage between 80% and 89%, and so on. You will get 0 base points for line coverage below 30%. The final points you get for line coverage of unit tests will be calculated as your base points multiplied by the percentage of your requirements coverage. For example, if you only implement 60% of the requirements and you achieve 95% line coverage in testing, you will get 6 = 10 * 60% points for this part.

6 Design Review

The 3-hour lecture time in Week 13 will be devoted to the peer review of your design. The review will be conducted in clusters of X groups (X is to be decided later); Each group needs to review the design of every other group in its cluster and fill out a review report. That is, each group will need to fill out and submit X-1 review reports, one for every other group in its cluster. In each report, the reviewer group needs to point out which design choices from the reviewee group are good, which are questionable, which should have been avoided, and explain the reasons.

Details about the design review's organization and the report's format will be announced before the review.

[2] https://www.jetbrains.com/help/idea/code-inspection.html
[3] http://en.wikipedia.org/wiki/Code_coverage
[4] https://en.wikipedia.org/wiki/Model-view-controller
[5] https://www.jetbrains.com/help/idea/code-coverage.html

7 Project Report

Each group must submit a project report that summarizes the group's work in the project, and you may refer to Section A for the required structure and contents for the report.

Individual contributions: In particular, your group needs to reach a consensus on the percentage of contributions each member has made to the project, and the percentages need to be clearly stated right below the report title. For example, if your group has 3 members and they have made equal contributions, you may write that each member has completed 33.3% of the work. Refer to Section 11 for how this percentage influences your individual score for the project.

If your group has problems in reaching such a consensus, you should submit all of the other required materials before the deadline and report your case to the instructor before 14:00, 22nd November 2026.

Learning-to-Learn: Some requirements in this project concern aspects of Java and/or object-oriented programming that are not fully covered in lectures, and you need to plan, conduct, evaluate, and adjust your self-learning activities to master the related knowledge and accomplish the corresponding tasks. Each group needs to 1) reflect on their learning-to-learn experiences in the project and 2) describe the plan to improve their self-learning approaches in a section named "Learning-to-Learn" of the final report.

8 Use of GenAI

Using GenAI tools in the project is allowed, but you should properly acknowledge all of the contents in your deliverables that were created with the help of those tools. More specifically, each group must fill out the "Honour Declaration for Group Project" and include it within the final submission, even if the group has not used GenAI in the project. You may refer to https://www.polyu.edu.hk/ar/docdrive/polyu-students/Student-guide-on-the-use-of-GenAI-revised-260521.pdf for more guidance on the use of GenAI.

9 Presentation

The presentation should not exceed 6 minutes, and each group member must present at least 1 minute.

Your presentation should at least include the following parts: 1) the overall architecture of your program, 2) a design choice for the program that concerns inheritance and polymorphism, and 3) a short discussion on how object-oriented design/programming helped you improve the reusability/scalability of the code. You may also include screen snapshots or short video clips of the Crobs running in your presentation, but do not let them dominate or excessively consume the overall presentation time.

Your presentation video should be in MP4 format. To facilitate validating the presenters, all presenters are required to show their student ID cards and faces at the beginning of the presentation, and the current presenter's face must always be shown in the video.[6] Each group member who does not show his/her face or student ID card in the presentation will have half of his/her marks deducted.

10 Submission

Each group is required to submit a single ZIP file containing the project code (including the source and test code), the user manual, the presentation slides and video, the project report, and the Honour Declaration for Group Project. Make sure that all of the documents you submit, unless specified otherwise, should be in PDF format. You should organize the files and directories in a

[6] All presentation videos will be automatically destroyed in the system after one year to protect your privacy.

consistent structure to ensure clarity and ease of access. Here are the guidelines for preparing the ZIP file:

1. Create a main directory with your group ID (and, optionally, your project's name or another suitable identifier).
2. Within the main directory, create two subdirectories for the ProjectCode and Presentation.
3. Place the relevant files in the required format in their corresponding directories. For example:
   - The project report should be placed inside the main directory.
   - The Honour Declaration for Group Project should be placed inside the main directory.
   - The user manual should be placed inside the main directory.
   - The IntelliJ IDEA project folder should be placed inside the ProjectCode directory.
   - The presentation slides and video should be placed inside the Presentation directory.
4. Select all of the directories and files within the main directory.
5. Compress the selected items into a ZIP file.

Example Directory Structure:

```
GroupID
├── project_report.pdf
├── Honour Declaration for Group Project.pdf
├── user_manual.pdf
├── ProjectCode
│   └── The IntelliJ IDEA project folder
└── Presentation
    ├── slides.pdf
    └── video.mp4
```

11 Grading

The overall project work of your group can earn at most 100 points in total from the following components:

- Requirements coverage (in total 40 points, as listed in Section 2)
- Code quality (10 points for good object-oriented design and 10 for good code style, as reported by the code inspection tool)
- Line coverage of unit tests (10 points)
- Presentation (7 points)
- Design review report (8 points)
- Individual report and user manual (15 points)
- Bonus points (9 points)

  Note: Bonus points can be used to reach, but not exceed, 100 points.

Your individual score for the project will be calculated based on the score for the overall project work of your group and your percentage of contributions to the project. Suppose the overall project work of a group is worth x points, the group has y members ($3 \leq y \leq 4$), and one member's contribution is $z\%$. The member's individual score for the project is calculated as $\min(x * y * z\%, x)$. We encourage equal contribution.

Failure to submit the group project Honour Declaration, or submitting a false declaration, will result in a penalty of up to 40% of the total points.

12 General Notes

- Java SE Development Kit Version 25[7] and IntelliJ IDEA (Version 2026.2.1[8]) will be used in grading your project. Make sure you use the same versions of tools in your development. Note that you need to configure "File | Project Structure... | Project Settings | Project" in the IntelliJ IDEA IDE as the following:
  - SDK: 25
  - Language level: 8 - Lambdas, type annotations etc.
- Your code should not rely on any library beyond the standard Java SE Development Kit 25 API.
- The project material package also includes three other files:
  - `Honour Declaration for Group Project.pdf`: Allows you to declare your use of GenAI, if any, in the project.
  - `SampleProject.zip`: Provides an example for the structure of the project directory. Feel free to build your system based on the sample project.
  - `COMP2021_PROJECT.xml`: Contains the rules for code inspection.
- If you use other tools and/or IDEs to program your system, make sure all your classes and tests are put into an IntelliJ IDEA project that is ready to be compiled and executed. **50 points will be deducted** from a project not satisfying this requirement or with compilation errors!

If you use other tools and/or IDEs to program your system, make sure all your classes and tests are put into an IntelliJ IDEA project that is ready to be compiled and executed. **50 points will be deducted** from a project not satisfying this requirement or with compilation errors!

[7] https://www.oracle.com/apac/java/technologies/downloads
[8] https://www.jetbrains.com/idea/download/?section=windows

Appendixes

A Project Report Template

Project Report
COMP2021 Object-Oriented Programming (Fall 2026)
Group XYZ

Members and contribution percentages:
Member1: a%
Member2: b%
Member3: c%
Member4: d%

1 Introduction

This report outlines the design and implementation of the Command Line Campus Room Booking System (Crobs), a project completed by Group XYZ for the course *COMP2021 Object-Oriented Programming* at PolyU.

2 The Command Line Campus Room Booking System (Crobs)

In this section, we describe first the overall design of the Crobs tool and then the implementation details of the requirements.

2.1 Design

Use class diagram(s) to give an overview of the system design and explain how different components fit together. Feel free to elaborate on the design patterns employed and/or justify critical design decisions to facilitate a comprehensive understanding of the system's construction.

2.2 Implementation of Requirements

For each (compulsory and bonus) requirement, describe 1) whether it is implemented and, when yes, 2) how you implemented the requirement as well as 3) how you handled various error conditions.

[REQ1] 1) The requirement is implemented.
2) Implementation details.
3) Error conditions and error handling.

[REQ2] 1) The requirement is not implemented.

[REQ3] ...

3 Learning-to-Learn

Use this section to 1) reflect on your learning-to-learn experiences in the project and 2) describe your plan to improve your self-learning approaches.

B User Manual

To prepare a user manual, it's important to note that there is no one-size-fits-all template. User manuals can differ significantly based on the system being documented and the specific needs of the users. However, the following guidelines can help you create an effective user manual:

- Introduction: Begin by providing an overview of the system and its purpose. Explain how the system works from a user's perspective.
- Commands Description: Describe all of the commands that the system supports. Provide detailed explanations of each command, including their functionalities and how they can be used.
- Screenshots: For each command, consider including screenshots to visually demonstrate the results under different inputs. Capture screenshots that showcase the system's response or output when specific commands are executed. Include captions or annotations to provide additional context or explanations for each screenshot.
- Step-by-Step Instructions: Provide clear and concise step-by-step instructions for users to follow when using each command. Break down the process into logical steps, ensuring that users can easily understand and replicate the actions.
- Troubleshooting: Include a troubleshooting section to address common issues or errors that users may encounter while using the system. Provide solutions or workarounds for each problem, ensuring that users can resolve issues on their own.
- Additional Resources: If applicable, include additional resources such as references, FAQs, or contact information for technical support. Provide users with avenues to seek assistance or further information if needed.
- Formatting and Organization: Ensure that the user manual is well-formatted and organized. Use headings, subheadings, and bullet points to improve readability. Consider using a consistent and intuitive structure throughout the manual.

Remember, the purpose of a user manual is to provide clear instructions and support to users. Tailor the manual to meet the specific needs of your system and its users.